FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de Tratamiento de Datos

Administración de Bases de Datos

Año: 2025

Seguridad en Bases de Datos

La seguridad en bases de datos es un pilar fundamental para proteger la información contra accesos no autorizados, garantizar la confidencialidad, integridad y disponibilidad de los datos, y cumplir con regulaciones como GDPR, HIPAA o PCI-DSS. Vamos a cubrir los principales aspectos de seguridad en bases de datos relacionales y no relacionales.

1. Usuarios, Roles y Servicios

La gestión de usuarios, roles y permisos es esencial para controlar quién puede acceder a la base de datos y qué operaciones puede realizar.

1.1. Autenticación

Proceso de verificar la identidad de un usuario o sistema. Métodos comunes incluyen:

- **Credenciales:** Nombre de usuario y contraseña.
- **Autenticación mutifactor (MFA):** Combina contraseñas con tokens, biometría o certificados.
- **Autenticación externa:** Uso de sistemas como LDAP, Active Directory o SSO (Single Sign-On) con OAuth o Kerberos.

Ejemplo en PostgreSQL:

```
CREATE USER usuario_lectura WITH PASSWORD 'contraseña_segura';
ALTER USER usuario_lectura WITH ENCRYPTED PASSWORD;
```

Mejores prácticas: Usar contraseñas fuertes, habilitar cifrado (SSL/TLS) para conexiones, y limitar intentos de inicio de sesión fallidos.

1.2. Autorización:

- Define los permisos específicos de un usuario o rol (lectura, escritura, eliminación, ejecución de procedimientos, etc.).
- Se implementa mediante sentencias como GRANT y REVOKE en SQL.

Ejemplo en MySQL:

```
GRANT SELECT, INSERT ON base_datos.tabla TO 'usuario_lectura'@'localhost';
REVOKE DELETE ON base_datos.tabla FROM 'usuario_lectura'@'localhost';
```

1.3. Control de Acceso Basado en Roles (RBAC):

- Asigna permisos a roles, no a usuarios individuales, para facilitar la administración.
- Ejemplo: Un rol "analista_datos" puede leer tablas, mientras que "admin_dba" tiene permisos completos.

Implementación en SQL Server:

```
CREATE ROLE analista_datos;
GRANT SELECT ON schema_datos.tabla_ventas TO analista_datos;
ALTER ROLE analista_datos ADD MEMBER usuario_analista;
```

- Ventajas: Escalable, reduce errores humanos, y simplifica auditorías.
- Mejores prácticas: Mínimo privilegio (otorgar solo los permisos necesarios) y revisión periódica de roles.

1.4. Cifrado de datos

El cifrado protege los datos contra accesos no autorizados, tanto en reposo (almacenados) como en transporte (durante la transmisión).

Cifrado en almacenamiento:

- Aplica algoritmos como AES-256 para cifrar datos en disco.
- Ejemplo: En Oracle, Transparent Data Encryption (TDE) cifra tablas o columnas sensibles.

Ejemplo en SQL:

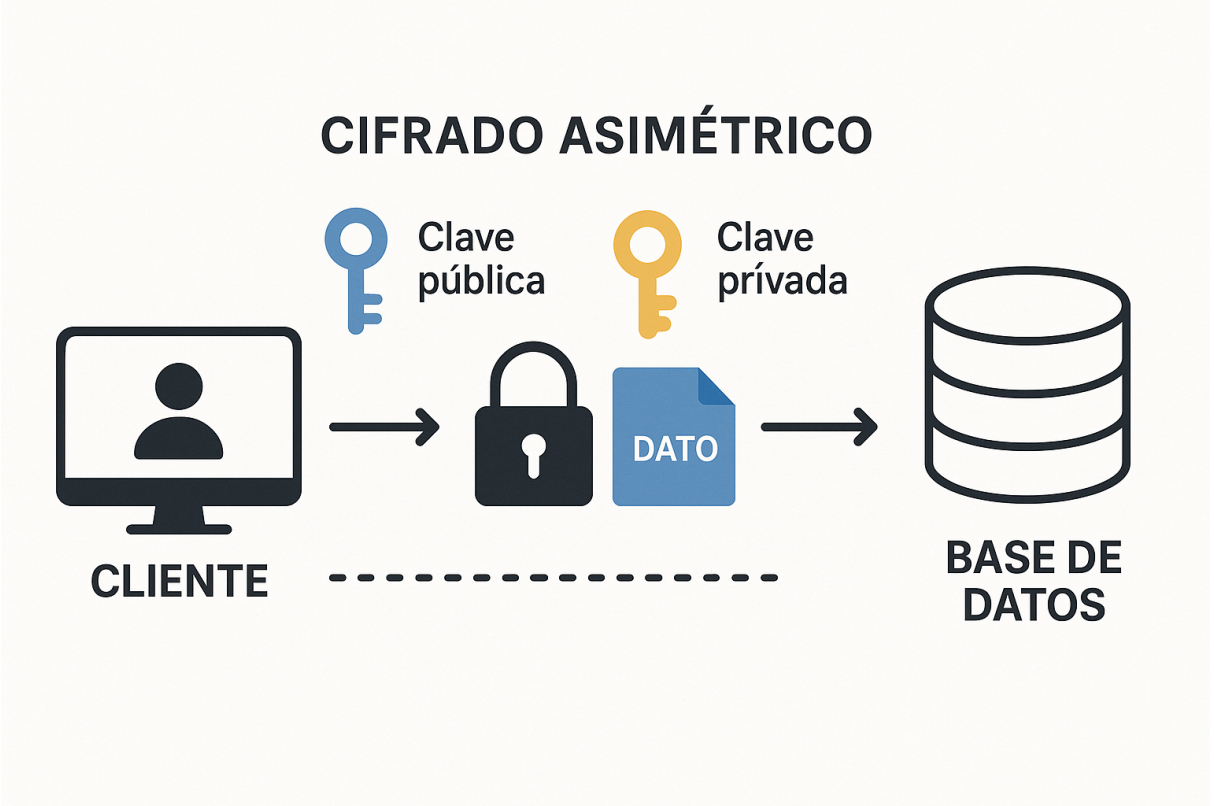
```
-- Habilitar TDE en Oracle
ALTER TABLE clientes ADD (numero_tarjeta VARCHAR2(16) ENCRYPT);
```

Herramientas: MySQL Enterprise Encryption, SQL Server TDE, o cifrado a nivel de sistema de archivos.

Cifrado en transporte:

Usa protocolos TLS/SSL para conexiones entre cliente y servidor.

Mediante el uso de criptografía con llaves públicas y privadas se establece el intercambio de datos entre cliente y servidor.



En SSL/TLS, el cifrado asimétrico se usa al inicio para intercambiar claves de forma segura. El servidor envía su clave pública, y el cliente la usa para cifrar una clave simétrica. Solo el servidor, con su clave privada, puede descifrarla.

Entre las mejores prácticas, se recomienda forzar a siempre usar conexiones SSL, mantener los certificados actualizados y evitar versiones obsoletas de TLS.

1.5. Auditoría de Bases de Datos

La auditoría registra las actividades en la base de datos para monitorear accesos, detectar anomalías y cumplir con normativas.

Componentes:

- Logs de actividad: Quién accedió, qué hizo, cuándo y desde donde
- Triggers: Registrar cambios en tablas específicas.
- Herramientas específicas: Oracle Audit Vault, SQL Server Audit o PostgreSQL pgAudit.

Ejemplo en PostgreSQL con pgAudit:

```
-- Habilitar auditoría de consultas SELECT
SET pgaudit.log = 'read';
```

Mejores prácticas:

- Almacenar logs en servidores separados para evitar manipulaciones.
- Configurar alertas para actividades sospechosas (por ej: múltiples intentos de acceso fallidos).
- Revisar normativas aplicables para definir qué auditar.

1.6. Copias de Seguridad y Recuperación

Las copias de seguridad (backups) y la recuperación son esenciales para garantizar la continuidad del negocio ante fallos, ataques o errores humanos.

Tipos de Backups:

- Completo: Copia de toda la base de datos.
- Incremental: Solo se procesan los cambios que se hayan hecho respecto al último backup.
- Diferencial: Cambios desde el último backup completo.

Estrategias de recuperación:

- Point in Time Recovery (PITR): Restaurar la base hasta un momento específico usando logs de transacciones.
- Restauración completa: Restaurar desde un backup completo.

Mejores prácticas:

- Realizar backups regulares (diarios, semanales)
- Almacenar copias en ubicaciones externas o en la nube.
- Probar procesos de recuperación periódicamente para validar su efectividad.
- Cifrar backups para proteger datos sensibles.

2. Concurrencia y Recuperación

La concurrencia y la recuperación son críticas para garantizar que múltiples usuarios o procesos puedan interactuar con la base de datos sin comprometer su integridad o consistencia.

2.1. Transacciones y Control de Concurrencia

Transacciones: Unidad lógica de trabajo que cumple las propiedades ACID:

- Atomicidad: Todas las operaciones se completan o ninguna (todo o nada).
- Consistencia: La base pasa de un estado válido a otro.
- Aislamiento: Las transacciones parecen ejecutarse de forma aislada.

- Durabilidad: Los cambios confirmados persisten incluso ante fallos.

Ejemplo en SQL

```
BEGIN TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

Si ocurre un error, se usa ROLLBACK para deshacer cambios.

Control de Concurrencia:

- Bloqueo (Locking): Restringe el acceso a datos mientras una trasacción los usa.
- Multiversión (MVCC): Cada transacción ve una "versión" de los datos. Esto suele ser usado en motores de bases de datos relacionales PostgreSQL y Oracle.
- Estampas de tiempo: Ordena transacciones según su marca temporal.

2.2. Integridad y Consistencia de los Datos

Restricciones de integridad:

Son reglas definidas para garantizar que los datos en una base de datos sean precisos, válidos y coherentes. Estas restricciones aseguran que los datos cumplan con las condiciones establecidas por el modelo de negocio. Los tipos principales incluyen:

- Integridad de dominio: Limita los valores permitidos en un atributo (ej. tipos de datos, rangos).
- Integridad de clave: Asegura que las claves primarias sean únicas y no nulas, y que las claves foráneas referencien valores válidos.

- Integridad referencial: Garantiza que las relaciones entre tablas sean consistentes (ej. no se puede eliminar un registro si es referenciado por otra tabla).

- Integridad definida por el usuario: Reglas específicas del negocio (ej. un campo EDAD debe ser mayor que 0).

Ejemplo SQL:

```
CREATE TABLE empleados (
    id INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    salario DECIMAL(10,2) CHECK (salario >= 0)
);
```

Consistencia:

Garantiza que los datos reflejen un estado lógico tras cada transacción. Por ejemplo, en una transferencia bancaria, el saldo total del sistema debe permanecer constante.

2.3. Problemas de concurrencia.

Cuando múltiples transacciones acceden a los mismos datos, pueden surgir problemas si no se controlan adecuadamente:

- Lecturas Sucias (Dirty Reads):

Una transacción lee datos no confirmados de otra transacción, que podría deshacerse.

> Ejemplo: T1 actualiza un saldo pero no hace commit; T2 lee el saldo modificado, pero T1 falla y revierte.

- Lecturas No Repetibles (Non-repeatable Reads):

Una transacción lee los mismos datos varias veces y obtiene valores diferentes proque otra transacción los modificó.

Ejemplo: T1 lee un saldo, T2 lo actualiza, T1 lee de nuevo y ve un valor diferente.

- Fantasmas (Phantom Reads):

Una transacción ejecuta una consulta repetidamente y ve filas nuevas o eliminadas por otra transacción.

Ejemplo: T1 consulta empleados con salario > 5000; T2 inserta un nuevo empleado; T1 repite la consulta y ve una fila adicional.

2.4. Control de concurrencia

Para mitigar los problemas de concurrencia, se utilizan varias técnicas:

- **Bloqueo (Locking)**

  Bloqueo exclusivo: Impide que otras transacciones lean o escriban en los datos bloqueados.

  Bloqueo compartido: Permite lecturas simultáneas, pero no escrituras.

  Ejemplo en SQL:

```
SELECT * FROM tabla WITH (UPDLOCK);
```

Su problema principal radica en que puede causar deadlocks (bloqueos mutuos), que requieren resolución automática o manual.

- **Estampas de tiempo (Timestamping):**

Asigna una marca temporal a cada transacción para determinar el orden de ejecución.

Si una transacción intenta modificar datos con una estampa posterior, se rechaza.

Ejemplo: Algoritmo de ordenación por timestamp en bases de datos distribuidas.

- **Serialización:**

Garantiza que las transacciones se ejecuten como si fueran secuenciales, aunque se procesen en paralelo.

Niveles de aislamiento en SQL:

  - Read Uncommited: Permite lecturas sucias.
  - Read Committed: Evita lecturas sucias.
  - Repeatable Read: Evita lecturas no repetibles.
  - Serializable: Evita fantasmas, garantizando máxima consistencia.

Ejemplo:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
SELECT * FROM pedidos WHERE cliente_id = 1;
COMMIT;
```

3. Desempeño (Rendimiento)

La optimización del rendimiento en bases de datos es crucial para garantizar tiempos de respuesta rápidos, escalabilidad y una experiencia de usuario eficiente, especialmente en sistemas con grandes volúmenes de datos o alta concurrencia. A continuación se mencionarán estrategias clave para mejorar el desempeño.

3.1. Estrategias de Indexación

Los índices mejoran la velocidad de las consultas al permitir que la base de datos localice datos rápidamente, aunque pueden añadir sobrecarga en operaciones de escritura.

Tipos de indices:

- Índices primarios:
    - Se crean automáticamente en la clave primaria de una tabla.
    - Garantizan unicidad y permiten búsquedas rápidas de registros específicos.
    - Ejemplo: Un índice en el campo ID de una tabla de usuarios.
- Índices secundarios:
    - Se crean en columnas no primarias para acelerar consultas frecuentes.
    - Pueden ser no únicos (permiten duplicados).
    - Ejemplo: Un índice en el campo apellido para buscar usuarios por apellido.
- Índices únicos:
    - Aseguran que los valores en una columna (o combinación de columnas) sean únicos.
    - Útiles para columnas como correos electrónicos o números de identificación.
    - Ejemplo: Índice único en email en una tabla de usuarios.
- Índices compuestos:
    - Cubren múltiples columnas para optimizar consultas que involucran combinaciones de campos.

    - Ejemplo: Índice en (ciudad, edad) para consultas que filtran por ambos criterios.
- Índices de texto completo (Full-Text):
    - Optimizan búsquedas de texto en columnas grandes, como descripciones o comentarios.
    - Útiles en aplicaciones con búsqueda de palabras clave.
    - Ejemplo: Índice en una columna de descripción para búsquedas de texto.
- Índices basados en estructuras de datos:
    - B-Tree: Ideal para rangos y comparaciones (>, <, =). Es el estándar en la mayoría de los sistemas.
    - Hash: Eficiente para búsquedas de igualdad exacta, pero no para rangos.
    - Bitmap: Útil para columnas con pocos valores distintos (ej. género) y consultas complejas.
    - Índices espaciales: Para datos geográficos, como coordenadas en mapas.
- Índices en clúster (Clustered vs. Non-Clustered):
    - Clustered: Organiza físicamente los datos en la tabla según el índice (solo uno por tabla, generalmente la clave primaria).
    - Non-Clustered: Estructura separada que apunta a los datos, permite múltiples índices por tabla.

Cuando usar índices:

- Columnas usadas frecuentemente en cláusulas WHERE, JOIN, GROUP BY u ORDER BY.

- Claves primarias y foráneas suelen indexarse automáticamente.
- Evitar índices en columnas con baja cardinalidad (pocoas valores distintos, ej: género) o tablas pequeñas.

Consideraciones:

- Los índices consumen espacio en disco y ralentizan operaciones de escritura (INSERT, UPDATE, DELETE).
- Mantener índices actualizados: Usar herramientas como REINDEX. En PostgreSQL u OPTIMIZE TABLE en MySQL.

Mejores prácticas:

- Indexar solo columnas críticas para consultas frecuentes.
- Monitorear índices no utilizadoss y eliminarlos.
- Usar índices de cobertura (covering indexes) para evitar accesos a la tabla principal.

3.2. Optimización de Consultas: Plan de Ejecución y Estadísticas.

La optimzación de consultas mejora el rendimiento al reducir el tiempo de ejecución y el uso de recursos.

- Plan de ejecución

  Es un mapa que muestra cómo la base de datos ejecuta una consulta (escaneos secuenciales, uso de índices, uniones, etc)

Componentes principales

- Operaciones: Incluyen escaneos de tablas (tables scan), búsqueda por índices (index seek), uniones (joins), ordenaciones (sorts), etc.
- Orden de ejecución: Define la secuencia en que se realizan las operaciones.
- Costo estimado: Indica el uso de recursos (CPU, memoria, I/O) para cada paso, ayudando a identificar cuellos de botella.

Tipos de planes

- Estimado: Generado sin ejecutar la consulta, basado en estadísticas.
- Real: Muestra el comportamiento real de la consulta tras ejecutarse, incluyendo filas procesadas y tiempos.

Ejemplo:

```
EXPLAIN ANALYZE SELECT * FROM pedidos WHERE cliente_id = 100;
```

Resultado:

```
Seq Scan on pedidos  (cost=0.00..35.50 rows=10 width=100) (actual
time=0.020..0.050 rows=8 loops=1)
  Filter: (cliente_id = 100)
  Rows Removed by Filter: 992
Planning Time: 0.150 ms
Execution Time: 0.070 ms
```

Explicación:

Elemento	Significado
Seq Scan on pedidos	Se realizó un escaneo secuencial sobre toda la tabla pedidos.

cost=0.00..35.50	Estimación de costo: desde el inicio hasta el final del escaneo.
rows=10	Estimación de filas que cumplen la condición.
width=100	Tamaño promedio de cada fila en bytes.
actual time=0.020..0.050	Tiempo real que tomó comenzar y terminar el escaneo.
rows=8	Cantidad real de filas que cumplieron la condición cliente_id = 100.
Loops=1	Número de veces que se ejecutó este paso (sola una vez en este caso)
Rows Removed by Filter: 992	Filas que no cumplieron la condición y fueron descartadas
Planning Time: 0.150 ms	Tiempo que tomó planificar la consulta
Execution Time: 0.070 ms	Tiempo total de ejecución

Componentes claves del plan

- Seq Scan: Escaneo secuencial (lento para tablas grandes)
- Index Scan: Uso de un índice (más rápido).
- Join Types: Nested Loop, Hash Join, Merge Join.

- Estadísticas:

Son datos que que la base de datos recopila sobre la distribución y características de los datos en tablas e índices. El optimizador usa estas estadísticas para generar planes de ejecución eficientes:

Tipos de estadísticas

- Distribución de valores: Indica la frecuencia de valores en una columna (histogramas).
- Cardinalidad: Número de valores únicos en una columna.
- Densidad: Proporción de duplicados en una columna o combinación de columnas.

Importancia:

- Ayudan al optimizador a estimar cuántas filas devolverá una consulta y elegir la mejor estrategia (ej. usar un índice o hacer un join)
- Estadísticas desactualizadas pueden llevar a planes ineficientes, como elegir un escaneo completo en lugar de un índice.

Actualizar estadísticas:

  - PostgreSQL: ANALYZE tabla;
  - MYSQL: ANALYZE TABLE tabla;

Entre los problemas comunes se pueden mencionar que las estadísticas desactualizadas pueden llevar a planes ineficientes.

3.3. Consejos y buenas prácticas de consultas SQL para un buen rendimiento.

Escribir consultas eficientes

- Seleccionar solo columnas escenciales: **evitar SELECT ***

```
-- Ineficiente
SELECT * FROM empleados;
-- Eficiente
SELECT id, nombre FROM empleados;
```

- Usar condiciones específicas: **Filtrar con WHERE lo antes posible.**

```
SELECT nombre FROM empleados WHERE departamento_id = 5;
```

- Evitar funciones en cláusulas WHERE: Funciones como UPPER() o TO_DATE() en columnas pueden impedir el uso de índices.

```
-- Ineficiente
SELECT * FROM pedidos WHERE UPPER(cliente) = 'JUAN';
-- Eficiente
SELECT * FROM pedidos WHERE cliente = 'Juan';
```

- Optimizar JOINs:
  - Usar INNER JOIN en lugar de LEFT JOIN cuando sea posible, ya que es más eficiente.
  - Asegurarse de que las columnas de unión estén indexadas.

```
SELECT e.nombre, d.nombre
FROM empleados e
INNER JOIN departamentos d ON e.departamento_id = d.id;
```

- Usar subconsultas con moderación: Reemplazar subconsultas correlacionadas por JOIN cuando sea posible.

```
-- Ineficiente
SELECT nombre FROM empleados WHERE EXISTS (
    SELECT 1 FROM departamentos WHERE id = empleados.departamento_id
);
```

```
-- Eficiente
SELECT e.nombre
FROM empleados e
INNER JOIN departamentos d ON e.departamento_id = d.id;
```

- Agrupaciones y Ordenaciones
  - Usar índices para columnas en GROUP BY u ORDER BY.
  - Minimizar el número de filas antes de agrupar con filtros en WHERE.

```
SELECT departamento_id, COUNT(*)
FROM empleados
WHERE activo = true
GROUP BY departamento_id;
```

Otras prácticas:

Evitar consultas anidadas profundas.

Usar vistas materializadas para datos calculados frecuentemente.

Monitorear y limitar el uso de recursos con parámetros como **work_mem** (PostgreSQL) o **query_cache_size** (MySQL)

3.4. Particionamiento y Sharding

Particionamiento:

Divide una tabla grande en partes lógicas (particiones) basadas en un criterio, como rangos de fechas o valores.

Tipos:

- **Por Rango:** Ej, dividir una tabla de pedidos por año.
- **Por Lista:** Ej, dividir por regiones geográficas.
- **Por Hash:** Distribuir datos uniformemente

Ejemplo en PostgreSQL (particionamiento por rango):

```
CREATE TABLE pedidos (
    id SERIAL,
    fecha DATE,
    cliente_id INT
) PARTITION BY RANGE (fecha);
CREATE TABLE pedidos_2023 PARTITION OF pedidos
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
```

Ventajas:

- Mejora el rendimiento de consultas al limitar el escaneo a particiones relevantes.
- Facilita la gestión de datos históricos (ej, archivar particiones antiguas).

Desventajas:

- Mayor complejidad en la gestión y posibles sobrecargas en operaciones de escritura.

Sharding:

Divide los datos físicamente entre múltiples servidores (shards), cada uno con un subconjunto de datos.

Ejemplo: Una base de datos de usuarios puede dividir los datos por región (un shard para Europa, otro para América).

Implementaciones:

- MySQL: Usar MySQL Cluster o herramientas como Vitess.
- PostgreSQL: Citus para sharding distribuido.

Ventajas:

- Escalabilidad horizontal: Añadir servidores para manejar más datos.
- Mejora el rendimiento en sistemas distribuídos

Desventajas:

- Complejidad en la gestión de consultas cruzadas entre shards
- Requiere planificación cuidadosa para el criterio de sharding (ej, por clave de cliente).

Mejores prácticas:

- Elegir un criterio de particionamiento/sharding basado en patrones de consulta.
- Combinar particionamiento y sharding para sistemas muy grandes.
- Monitorear el balanceo de datos entre shards para evitar puntos calientes (hotspots).

4. Modelos de Procesamiento

Definen cómo una base de datos organiza, accede y procesa datos en respuesta a solicitudes de usuarios o aplicaciones.

Características clave:

- **Escalabilidad:** Capacidad de manejar mayores volúmenes de datos o usuarios.

- **Consistencia:** Garantizar que los datos sean coherentes en todo el sistema.
- **Disponibilidad:** Mantener el sistema operativo incluso ante fallos.
- **Latencia:** Minimizar el tiempo de respuesta para consultas.

Tipos de modelos:

- Centralizado: Una sola base de datos en un servidor.
- Distribuido: Datos y procesos distribuidos en múltiples nodos.
- Híbrido: Combina elementos centralizados y distribuidos.

4.1. Modelo Cliente-Servidor

Arquitectura donde un cliente (aplicación o usuario) envía solicitudes a un servidor de base de datos, que procesa y responde.

Ejemplo: Una aplicación web (cliente) consulta un servidor MySQL para obtener datos de usuarios.

Componentes:

- Cliente: Envía consultas SQL o comandos a través de una conexión (ODBC, JDBC, etc).
- Servidor: Ejecuta consultas, gestiona datos y devuelve resultados.
- Protocolo: TCP/IP, sockets o APIs específicas.

Ventajas:

- Centralización de la lógica de procesamiento en el servidor.

- Facilidad de mantenimiento y actualizaciones.

Desventajas:

- Cuello de botella en el servidor si hay muchas conexiones.
- Latencia en redes lentas.

4.2. Distribución de datos

Divide los datos entre mútiples nodos físicos o lógicos para mejorar escalabilidad y disponibilidad.

Técnicas:

- Replicación: Copiar datos en múltiples nodos.
    - Maestro-Esclavo: Un nodo maestro escribe, los esclavos leen (ej, MySQL Replication).
    - Multimaster: Múltiples nodos permiten escritura (ej, PostgreSQL con BDR).
- Particionamiento: Dividir datos en particiones lógicas.
- Sharding: Distribuir datos físicamente entre servidores.

Ventajas:

- Mejora el rendimiento al distribuir la carga.
- Mayor tolerancia a fallos mediante redundancia.

Desventajas:

- Complejidad en mantener la consistencia.
- Consultas cruzadas entre nodos pueden ser lentas.

4.3. Distribución de procesos

Divide las tareas de procesamiento (consultas, cálculos) entre múltiples nodos para paralelizar el trabajo.

Técnicas:

- Procesamiento paralelo: Divide una consulta en subconsultas ejecutadas en paralelo.
- Computación distribuida: Usa frameworks para procesar datos en clústeres.

Ventajas:

- Reduce el tiempo de ejecución para consultas complejas.
- Escala horizontalmente al añadir nodos.

Desventajas:

- Requiere infraestructura robusta.
- Complejidad en la sincronización de procesos.

4.4. Two-Phase Commit (2PC)

Protocolo para garantizar la consistencia en transacciones distribuidas entre múltiples nodos.

Fases:

- Fase 1 (Preparación): El coordinador pide a todos los nodos que preparen la transacción y confirmen si pueden hacer commit.

- Fase 2 (Confirmación): Si todos los nodos están listos, el coordinador ordena el commit. Si algo falla, se realiza un rollback.

Ejemplo:

En una base bancaria distribuida, se realiza una transferencia entre dos nodos.

1. Nodo A prepara deducción del saldo.
2. Nodo B prepara adición del saldo.
3. Si ambos confirman, se realiza el commit, sino el rollback.

Ventajas:

- Garantiza consistencia en sistemas distribuidos.

Desventajas:

- Puede ser lento debido a la coordinación.
- Riesgo de bloqueos si un nodo falla.

4.5 Propagación Asincrónica

Los cambios en uno nodo se propagan a otros nodos de forma no inmediata, permitiendo mayor disponibilidad a costa de consistencia temporal.

Como ejemplo podemos mencionar que el nodo maestro recibe cambios y lo replica a los nodos esclavos.

Ventajas:

- Mejora la disponibilidad y reduce la latencia en escrituras.
- Ideal para sistemas donde la consistencia eventual es aceptable (ej: redes sociales).

Desventajas:

- Posibles inconsistencias temporales (lecturas desactualizadas).
- Complejidad en la resolución de conflictos.